Ejercicio práctico 1.2

«La biblioteca del barrio»

Nombre	Julian Zuñiga,Bryam Villalva
Curso	3 ciclo desarrollo de software
Fecha	1571072026

Asignatura	Herramientas informáticas para el despliegue de diagramas
Unidad	1. Principios de la arquitectura web en el diseño y desarrollo de aplicaciones backend
Tema	1.2 Métodos HTTP y operaciones sobre recursos
Docente	Jhostin Omar Vacacela Saca
Duración	60 minutos, dentro de las 4 horas de clase
Modalidad	En parejas

Objetivo

Elegir el método HTTP adecuado para cada operación sobre un recurso, implementarlo en Nest.js y analizar la respuesta del servidor.

Situación

La biblioteca de un barrio quiere pasar su catálogo de fichas de papel a un servicio web. Cada libro tiene un id, un título, un autor y un campo disponible que vale true cuando el libro está en el estante.

Tu pareja y tú construyen el servicio que expone ese catálogo. El recurso se llama libros y vive en la dirección /libros.

Instrucciones generales

1. Trabaja con tu pareja. Las partes A a C se resuelven en este mismo documento; la parte D se resuelve en código.
2. Entrega un archivo .zip con este documento respondido, la carpeta del proyecto sin node_modules, la colección de Postman exportada en .json y la captura de las pruebas en verde.
3. Para ejecutar el proyecto usa Node.js 22 o superior: npm install y luego npm start.
4. Si usas inteligencia artificial, decláralo en la parte E. Sin declaración, la parte E no puntúa.

Parte A. Leer y predecir (20 puntos)

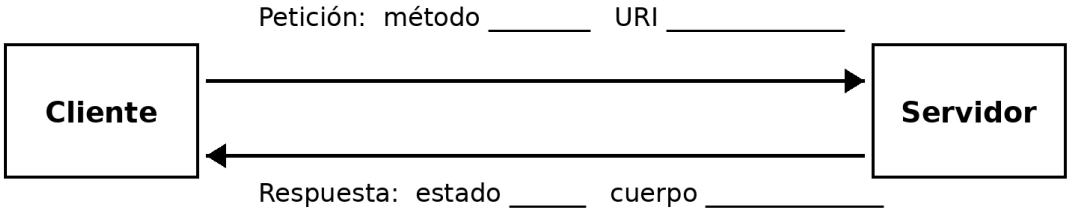
El servicio recién iniciado tiene estos tres libros y la variable siguienteId vale 4:

```
{ id: 1, titulo: "Cien años de soledad", autor: "García Márquez", disponible: true }
{ id: 2, titulo: "El túnel",             autor: "Sabato",          disponible: true }
{ id: 3, titulo: "Rayuela",              autor: "Cortázar",        disponible: false }
```

Y este es el código de dos de sus endpoints:

```
@Get(':id')
obtener(@Param('id', ParseIntPipe) id: number) {
  return this.libros[this.indiceDe(id)];
  // si no existe, indiceDe lanza NotFoundException(`No existe el libro ${id}`)
}

@Post()
crear(@Body() cuerpo, @Res({ passthrough: true }) res) {
  const nuevo = { id: this.siguienteId++, ...cuerpo };
  this.libros.push(nuevo);
  res.setHeader('Location', `/libros/${nuevo.id}`);
  return nuevo;
}
```



Dibuja o escribe aquí lo que predices en cada pregunta.

Las peticiones se envían en este orden, una tras otra, sobre el servicio recién iniciado. Escribe el estado y el cuerpo que devolverá cada una.

A1 · 5 puntos
GET /libros/2
Tu respuesta

Estado: 200 ok

Cuerpo:

```
  "id": 2,
 "título": "El túnel",
 "autor": "Sabato",
 "disponible": true
```

A2 · 5 puntos

GET /libros/9

Tu respuesta

Estado:404 no found No existe el libro ${id}`

Cuerpo:

```
"statusCode": 404,
 "message": "No existe el libro 9",
 "error": "Not Found"
```

A3 · 5 puntos

POST /libros con el cuerpo {"titulo":"Pedro Páramo","autor":"Rulfo","disponible":true}

Tu respuesta

Estado:201 Created

Cabecera Location:/libros/4

Cuerpo:

```
{
 "titulo": "Pedro Páramo",
 "autor": "Rulfo",
 "disponible": true
}
```

A4 · 5 puntos

La misma petición A3, enviada otra vez. ¿Qué estado y qué Location devuelve? ¿Cuántos libros hay en el catálogo después de A4?
Tu respuesta

Estado:201 Created
Cabecera Location:/libros/5
Libros en el catálogo:
{
"id": 5,
"titulo": "Pedro Páramo",
"autor": "Rulfo",
"disponible": true
}

Parte B. Completar (20 puntos)

Completa los cinco espacios en blanco del controlador. Cada espacio vale 4 puntos. Los comentarios dicen qué debe hacer cada función.

```
@Controller('libros')
export class LibrosController {
  @(1)_____()                          // lista la colección
  listar(@Query('autor') autor?: string) { /* ... */ }

  @(2)_____(':id')                     // reemplaza el libro completo
  reemplazar(@Param('id', ParseIntPipe) id: number, @Body() cuerpo) { /* ... */ }

  @(3)_____(':id')                     // cambia solo los campos enviados
  modificar(@Param('id', ParseIntPipe) id: number, @Body() cambios) { /* ... */ }

  @(4)_____(':id')                     // elimina el libro
  @HttpCode(HttpStatus.(5)_____)        // responde sin cuerpo
  eliminar(@Param('id', ParseIntPipe) id: number) { /* ... */ }
}
```

Espacio	Tu respuesta
(1)	GET
(2)	PUT
(3)	PATCH
(4)	DELETE
(5)	NO_CONTENT

Parte C. Detectar y corregir errores (20 puntos)

Cada situación tiene un error en la elección del método o de la dirección. Explica cuál es y escribe cómo lo corregirías. Cada una vale 5 puntos.

C1 · 5 puntos
Un compañero escribió: @Get(':id/borrar') borrar(...) { /* elimina el libro */ }
Tu respuesta
Error:Uso el metodo incorrecto Corrección:Usar el método DELETE

C2 · 5 puntos
Otro escribió: @Put(':id') marcarPrestado(@Param('id') id, @Body() cuerpo: { disponible: boolean }) { /* guarda solo el campo disponible */ }
Tu respuesta
Error:Usó el método PUT el cual actualiza todo Corrección:Necesita Usar el método PATCH

C3 · 5 puntos
Un tercero escribió, dentro de @Controller('libros'): @Post('crearLibro') crear(...) { /* crea un libro */ }
Tu respuesta

Error:Está usando crear libro lo que genera una ruta equivocada POST /libros/crearLibro
Corrección:Debería usar @Controller('libros')
@Post()
crear(...) {
/* crea un libro */
}
Así la ruta seria POST /libros

C4 · 5 puntos

Alguien probó PUT /libros/1 con el cuerpo {"disponible":false} para marcar el libro como prestado y recibió 200 con este cuerpo: {"id":1,"disponible":false}. ¿Qué pasó con el título y el autor? ¿Qué petición debió enviar?
Tu respuesta
Qué pasó:Usó mal el método PUT y terminó remplazando el contenido del libro de id:1 a {"disponible":false} Petición correcta:Debería usar PATCH para actualizar solo el estado {"disponible":false}, y poder cambiar el estado del libro.

Parte D. Escribir desde cero (30 puntos)

Implementa el servicio de la biblioteca. Usa como modelo el proyecto api-productos visto en clase.

1. Copia api-productos y renómbralo libros-api. Cambia el recurso a libros con los campos id, titulo, autor y disponible, y carga los tres libros de la parte A.
2. Crea los seis endpoints de la tabla. Si el id no existe, el servicio debe responder 404.
3. En Postman crea una colección «Biblioteca» con una petición por endpoint. En cada una escribe una prueba que compruebe el estado esperado.
4. Ejecuta la colección completa y guarda una captura con todas las pruebas en verde.

Operación	Método y dirección	Estado esperado	Puntos
Listar, con filtro opcional ?autor=	`GET /libros`	200	5
Obtener un libro	`GET /libros/{id}`	200, o 404 si no existe	5
Crear un libro (envía Location)	`POST /libros`	201	5
Reemplazar un libro completo	`PUT /libros/{id}`	200	5
Cambiar campos de un libro	`PATCH /libros/{id}`	200	5
Eliminar un libro	`DELETE /libros/{id}`	204	5

Criterio de éxito: las seis pruebas de la colección pasan en verde y puedes explicar, para cada petición, por qué elegiste ese método.

Parte E. Reflexión y declaración de uso de IA (10 puntos)

E1 · 6 puntos
Un cliente móvil pierde la señal justo después de enviar POST /libros y reintenta solo. ¿Qué puede pasar en el catálogo? ¿Podría reintentar sin riesgo un DELETE /libros/3? Apóyate en lo que viste en las partes A y D.

Tu respuesta
1. Podría duplicarse los libros al reintentar. o si el server no lo creo a la primera lo crea en el reintento. 2. Si podría usar DELETE ya que este es un método idempotente.Si la primera petición elimina el libro 3, los reintentos posteriores no volverán a eliminar más libros ni producirán efectos adicionales sobre el catálogo. El resultado final sigue siendo que el libro 3 ya no existe.

Declaración de uso de inteligencia artificial · 4 puntos
Marca una opción: [ ] No usé IA. [x] Usé IA. Herramienta usada: Chat Gpt Para qué la usé: Para guiarme en comandos,codigos de nest y postman Qué revisé y corregí yo: instalar las dependencias de node y nest

Se permite el uso de IA con declaración de la herramienta y el alcance. Como referencia, el reporte de Turnitin no debe pasar de 30 % de contenido posiblemente generado por IA ni de 10 % de similitud.

Evaluación

Instrumento del PEA para este tema: trabajo práctico de codificación. Se califica con la siguiente lista sobre 100 puntos.

Parte	Qué se evalúa	Puntos
A	Predice el estado, las cabeceras y el cuerpo de cada petición (4 × 5)	20
B	Elige el decorador y el estado correctos (5 × 4)	20
C	Detecta el error y propone el método o la dirección adecuados (4 × 5)	20
D	Cada endpoint funciona y su prueba de Postman pasa en verde (6 × 5)	30
E	Reflexión (6) y declaración de uso de IA (4)	10
Total		100

Lista de cotejo de entrega

1. El .zip incluye este documento, el proyecto sin node_modules, la colección de Postman y la captura.

2. El proyecto arranca con npm install y npm start sin pasos extra.
3. La colección tiene seis peticiones y todas sus pruebas pasan en verde.
4. La declaración de uso de IA está completa.

Referencias

Mozilla Developer Network. (s. f.). HTTP request methods. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). HTTP semantics (RFC 9110). IETF. https://www.rfc-editor.org/rfc/rfc9110

NestJS. (s. f.). Controllers. https://docs.nestjs.com/controllers